



Gymnázium J. K. Tyla

Úvod do agentického AI a design autonomní pipeline pro softwarový vývoj

Návrh a realizace systému DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Tato odborná práce představuje návrh, implementaci a provozní ověření systému DarkFactory — modulární platformy pro autonomní vývoj softwaru na GitHubu. Systém přebírá celý životní cyklus vývojového požadavku od zadání v GitHub Issues, přes interpretaci a technické plánování, až po generování kódu, běh testů a vystavení pull requestu. Důraz je kladen na lidskou kontrolu prostřednictvím dvoustupňového schvalování (Human Gate) a na spolehlivost provozu zajištěnou automatickou rotací modelů při vyčerpání API kvót a bezetrátovým ukládáním stavu. Celá infrastruktura je sdílena napříč repozitáři formou znovupoužitelných workflow a řízena jediným manifestem (.github/darkfactory.json). Systém byl úspěšně nasazen na tři repozitáře zahrnující vývoj v Pythonu i sazbu této práce v systému Typst, přičemž dosáhl 86,9% úspěšnosti dokončení úloh.

Klíčová slova

autonomní systémy, softwarové inženýrství, kontinuální integrace, orchestrace agentů, jazykové modely

Annotation

This thesis presents the design, implementation, and operational evaluation of DarkFactory, a modular platform for autonomous software engineering on GitHub. The system automates the complete lifecycle of an engineering request: from issue intake, semantic interpretation, and technical planning, to code generation, testing, and pull request delivery. Governance is maintained through a two-gate human approval contract, while operational reliability is ensured via automated model and account rotation upon API quota exhaustion and lossless state checkpointing. The entire pipeline is shared across repositories via reusable workflows and governed by a single configuration manifest (.github/darkfactory.json). The system was deployed across three repositories covering Python software and the typesetting of this thesis in Typst, achieving an 86.9% completion rate.

Keywords

autonomous systems, software engineering, continuous integration, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace	7
1.2 Cíl práce	7
1.3 Metodika	8
2 Teoretická část	9
2.1 Řízení verzí	9
2.1.1 Větve a jejich role	9
2.1.2 Model pull requestu	9
2.2 Kontinuální integrace	10
2.2.1 Požadované kontroly	10
2.2.2 Sestavení a artefakty	10
2.3 Jazykové modely a jejich orchestrace	10
2.3.1 Transformer a LLM	10
2.3.2 Tokeny, tokenizér a embedding	11
2.3.3 Mechanismus pozornosti a jeho typy	13
2.3.4 Multimodální modely	17
2.3.5 Context	18
2.3.6 Prompt a kontextové inženýrství	21
2.3.7 Agent vs Chatbot	22
2.3.8 Harness a System Prompt	22
2.3.9 ReAct smyčka	22
2.3.10 Vyvolávání nástrojů	24
2.3.11 Dovednosti, skripty a zachytné body	25
2.3.12 MCP Server	26
2.3.13 Meta Harness	26
2.3.14 Subagenti	26
2.3.15 Workflows neboli Graph Engineering	27
2.4 Human in the loop neboli člověk ve smyčce	28
2.4.1 Schvalovací body	28

2.4.2	Dohledatelnost	28
2.4.3	Eskalace a transparence selhání	28
3	Praktická část	29
3.1	Cíl a rozsah systému	29
3.2	Architektura	29
3.2.1	Koncepce a systémové vrstvy	29
3.2.2	Sdílení pracovních postupů	29
3.2.3	Konfigurační model repozitáře	29
3.3	Životní cyklus požadavku	30
3.4	Popis prostředí repozitáře	30
3.4.1	Domény a prostředí	30
3.4.2	Deklarace jako doplněk rozpoznávání	30
3.5	Orchestrace napříč poskytovateli	31
3.6	Ověřování změn	31
3.7	Projektová automatizace a správa nastavení jako kód	31
3.8	Automaticky generovaná dokumentace	32
3.9	Systém revizních značek pro lidský dohled nad akademickým textem	32
4	Výsledky a diskuse	34
4.1	Profil nasazených repozitářů a testovací prostředí	34
4.2	Sjednocení pracovních postupů	34
4.3	Doménové ověření na sazbě textu	35
4.4	Provozní metriky a spolehlivost agentních běhů	35
4.5	Poznatky a analýza chyb z provozu	35
4.6	Diskuse	36
4.7	Omezení	36
5	Závěr	38
	Seznam příloh	40
A	Obsah přiloženého média	41
B	Schéma konfiguračního manifestu darkfactory.json	42
C	Sdílené workflow pro GitHub Actions	43
D	Systémové prompty plánovacího a kódovacího agenta	44
E	Protokol revizních značek v sazebním systému Typst	45
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	45
	Seznam zdrojů	47

Seznam obrázků a tabulek

Obrázek 1	Geometrická reprezentace sémantických vztahů v embeddingovém prostoru: A) Klasická vektorová aritmetika pojmů ($\vec{v}(\text{král}) - \vec{v}(\text{muž}) + \vec{v}(\text{žena}) \approx \vec{v}(\text{královna})$), B) Izomorfní algebraické relace v programování (funkce $\rightarrow$ metoda, proměnná $\rightarrow$ atribut).....	12
Tabulka 1	Empirické porovnání tokenové náročnosti ekvivalentního větného významu v angličtině a češtině (tokenizér <code>cl100k_base</code>).....	13
Tabulka 2	Srovnání architektur organizace hlav mechanismu pozornosti a jejich dopad na paměťovou náročnost KV cache.	16
Obrázek 2	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	23
Tabulka 3	Rozpoznávání prostředí podle souboru popisujícího balíček.	30

1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi okamžikem, kdy někdo popíše požadavek, a okamžikem, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Nejde o představu úplného vyloučení člověka — i temná továrna má konstruktéry, kteří rozhodují, co se bude vyrábět — nýbrž o vyloučení člověka z opakujících se úkonů. Tato práce zkoumá, jak stejný princip uplatnit ve vývoji softwaru.

1.1 Motivace

S rozšířením velkých jazykových modelů se objevila možnost automatizovat i psaní kódu. Většina dostupných nástrojů však řeší jen dílčí krok: vygenerují návrh změny, který někdo musí zasadit do procesu, ověřit a schválit. Chybí popis toho, jak má takový nástroj zapadnout do vývojového procesu jako celku — kdo schvaluje, co se stane při selhání a jak se pozná, že je výsledek správný.

Právě tato mezera je předmětem práce. Zajímavá není otázka, zda model dokáže napsat kód; to je dnes doloženo. Zajímavá je otázka, jaké okolí musí kolem takového modelu vzniknout, aby jeho výstupu bylo možné důvěřovat.

1.2 Cíl práce

Cílem této práce je navrhnout, realizovat a ověřit systém, který automatizuje vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a orchestrace jazykových modelů.
2. Navrhnout architekturu systému, který provede požadavek celým procesem.
3. Systém realizovat a nasadit na reálné repositáře.
4. Vyhodnotit jeho chování a pojmenovat omezení, na která v provozu narazil.

V návaznosti na stanovené cíle si práce klade tři konkrétní inženýrské výzkumné otázky:

- **VO1 (Míra automatizace a role člověka):** Lze vývojový proces od zadání požadavku (GitHub Issue) po vytvoření funkčního pull requestu plně zautomatizovat tak, aby role vývojáře spočívala pouze ve schvalování záměru a plánu, aniž by musel sám psát kód nebo řešit syntaktické chyby?

- **V02 (Doménová přenositelnost):** Lze identické centrální workflow a tutéž agentní pipeline použít pro programovací kód (Python) i pro sazbu textového dokumentu (Typst) pouhou změnou jediného konfiguračního souboru (`.github/darkfactory.json`)?
- **V03 (Provozní odolnost a obnova):** Dokáže systém samostatně překonat vyčerpání API limitů (HTTP 429) a chyby v testech, aniž by došlo k havárii běhu v GitHub Actions nebo ke ztrátě rozpracovaného kódu?

Odpovědi na tyto otázky jsou v práci ověřeny empirickým provozním nasazením na třech produkčních repozitářích, vyhodnocením úspěšnosti průchodu životním cyklem požadavků a analýzou chování systému při simulovaných i reálných výpadech infrastruktury.

1.3 Metodika

Práce je z povahy tématu konstrukční a inženýrská: primárním výstupem je funkční, plně integrovaný systém a empirické vyhodnocení jeho provozní spolehlivosti v reálném vývoje-
vém prostředí. Postup odpovídá iterativnímu inženýrskému cyklu: po analýze teoretických východisek následoval návrh modulární architektury, implementace řídicího metaharnesu a jeho postupné nasazení na tři typově odlišné repozitáře:

1. **DarkFactory:** mateřský repozitář systému (Python, GitHub Actions, metaharness).
2. **OdbornaPrace-paper:** repozitář samotného rukopisu této práce (doména akademických textů a sazby v systému Typst).
3. **ChessWithQuests:** aplikační projekt s herní logikou.

Empirické ověření probíhalo longitudinálním sledováním reálných integračních běhů v prostředí GitHub Actions nad skutečnými požadavky (GitHub Issues) a pull requesty. Místo syntetických laboratorních benchmarků (např. izolovaného vyhodnocování na datasetech typu SWE-bench) se výzkum soustředil na end-to-end spolehlivost v produkčních podmínkách: sledována byla schopnost pipeline projít celým životním cyklem bez uvážnutí, četnost vyčerpání kontextu či API limitů, chování záchranných mechanismů při rotaci modelů a zejména kvalitativní a kvantitativní analýza chyb, které se projeví v reálném provozu. Získané poznatky sloužily k průběžné optimalizaci a zpevnění mantinelů celého systému.

2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a princip zapojení člověka do smyčky (*Human-in-the-loop*). Cílem není vyčerpávající přehled, nýbrž zavedení pojmů v podobě, v jaké s nimi pracuje navržený systém.

2.1 Řízení verzí

Systém pro řízení verzí uchovává historii změn zdrojového kódu. Distribuovaný model, jehož nejrozšířenějším zástupcem je Git, se od centralizovaného liší tím, že každý vývojář má úplnou kopii historie (1). Změny lze proto vytvářet a zkoumat i bez spojení se serverem a slučovat je až ve chvíli, kdy jsou hotové.

2.1.1 Větev a jejich role

V moderních distribuovaných systémech správy verzí (zejména v systému Git (1)) je repositář formálně modelován jako orientovaný acyklický graf (*Directed Acyclic Graph*, DAG), jehož uzly tvoří neměnné objekty revizí (*commits*) provázané kryptografickými hashy (SHA-1 či SHA-256) na své rodičovské stavy. Větev v tomto modelu nepředstavuje fyzickou kopii souborů, nýbrž odlehčený, pohyblivý ukazatel (*ref*) na konkrétní uzel grafu.

Tato grafová architektura má klíčový význam pro bezpečné zapojení autonomních agentů do vývojového procesu:

- **Izolace stavu:** Agent operuje ve vyhrazené větvi (např. `feature/...` nebo `agent/...`), která odbočuje z hlavní vývojové linie (`main`). Veškeré pokusné mutace souborového systému, mezistavy a ladící kroky zůstávají striktně izolované, aniž by ohrozily stabilitu produkčního kódu nebo práci ostatních členů týmu.
- **Deterministický audit:** Každý krok agenta lze reprezentovat jako atomický commit s přesným časovým otiskem, autorskými metadaty a odkazem na kontextové zadání. Vzniká tak neměnná a zpětně ověřitelná historie změn.
- **Strategie slučování:** Při integraci hotové větve do hlavní linie se uplatňují různé topologické strategie: přímý posun ukazatele (*fast-forward*), vytvoření explicitního slučovacího uzlu (*merge commit*), nebo sloučení celé sekvence dílčích mezikroků agenta do jediného čistého uzlu (*squash and merge*). Právě squashování je v autonomních pipeline preferováno, neboť eliminuje šum v podobě neúspěšných pokusů modelu a v hlavní větvi zanechává pouze finální, ověřený přírůstek.

2.1.2 Model pull requestu

Sloučení větve do hlavní vývojové linie se v moderním kolaborativním softwarovém inženýrství odehrává prostřednictvím modelu **pull requestu** (PR, na platformě GitLab též

Merge Request). Jde o formalizovaný procesní uzel, v němž autor větve předkládá navržený diff kódu k revizi dříve, než dojde k jeho trvalému začlenění do chráněné hlavní větve. V kontextu autonomního vývoje plní pull request dvě nezastupitelné funkce:

1. **Strojová validační brána:** Na vytvoření nebo aktualizaci PR reaguje integrační server (CI), který v izolovaném kontejneru spustí sadu automatizovaných testů, typových kontrol a bezpečnostních linterů. Tím je objektivně ověřeno, že syntetický kód generovaný modelem splňuje stanovené standardy a nezpůsobuje regresi stávající funkcionality.
2. **Lidská schvalovací brána (Human Gate):** Pull request poskytuje přehledné rozhraní zobrazující řádkový diff změn, výsledky automatických testů a strukturovaný popis úprav. Člověk v roli recenzenta (*code reviewer*) tak může provést finální sémantickou kontrolu a rozhodnout o schválení či zamítnutí změny. Pull request tím představuje ideální architektonický styčný bod pro princip člověka ve smyčce (*Human-in-the-loop*).

2.2 Kontinuální integrace

Kontinuální integrace (angl. *continuous integration*) je praxe, při níž se každá změna automaticky sestaví a otestuje (2). Namísto dlouhých období, kdy se změny hromadí a slučují až na konci, se ověřuje průběžně a v malých dávkách, takže chyba je odhalena blízko svému vzniku.

2.2.1 Požadované kontroly

Ke kontinuální integraci patří pojem *požadovaných kontrol* (angl. *required checks*): množina úloh, které musí skončit úspěšně, jinak nelze změnu sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli pouze dohodou mezi vývojáři.

Návrh požadovaných kontrol má jedno nezřejmé úskalí. Úloha, která se za jistých okolností „přeskočí“, nehlásí žádný výsledek; je-li přitom uvedena mezi požadovanými, zablokuje slučování napořád. Kontrola proto musí vždy skončit nějakým závěrem — i kdyby jím bylo konstatování, že v daném repozitáři není co dělat.

2.2.2 Sestavení a artefakty

Výsledkem sestavení bývá *artefakt*: spustitelný soubor, knihovna, nebo — jak ukazuje praktická část — vysázený dokument. Automatizace vydávání verzí spojuje artefakt se značkou v historii, takže ke každé vydané verzi existuje doložitelný výstup.

2.3 Jazykové modely a jejich orchestrace

2.3.1 Transformer a LLM

Transformer je architektura hlubokých neuronových sítí představená společností Google v roce 2017 v rámci výzkumu strojového překladu. Na rozdíl od starších architektur (např. rekurentních sítí RNN), které sekvence zpracovávaly krok po kroku a trpěly ztrátou dlouhodobého kontextu, využívá transformer mechanismus zvaný *attention* (pozornost).

Ten modelu umožňuje paralelně vyhodnocovat sémantické souvislosti mezi všemi tokeny v sekvenci bez ohledu na jejich vzájemnou vzdálenost.

LLM neboli *Large Language Model* (velký jazykový model) je rozsáhlá neuronová síť postavená na architektuře transformeru a předtrénovaná na textových datech o objemu stovek miliard až bilionů tokenů. Díky mechanismu pozornosti model dokáže zachytit hluboké syntaktické i sémantické struktury přirozeného jazyka, což se navenek projevuje schopností generalizace, abstrakce a generování korektního zdrojového kódu.

Tento architektonický přelom popsala publikace *Attention Is All You Need* (3). Původní model byl koncipován jako **Encoder-Decoder** (kodér-dekodér) pro strojový překlad: obousměrný kodér nejprve zkomprimoval celou vstupní větu a dekodér za pomoci křížové pozornosti (*cross-attention*) generoval překlad v cílovém jazyce.

Ve vývoji softwaru a moderních agentních systémech se však naprostým standardem stala architektura **Decoder-only** (např. GPT, Claude, LLaMA či DeepSeek). Tyto modely pracují čistě autoregresivně — predikují vždy následující nejpravděpodobnější token na základě celého předcházejícího kontextu. Instrukce, pravidla, kontext repozitáře i rozepsaný kód tvoří jedinou společnou sekvenci, což umožňuje plynulé doplňování kódu i přímé generování volání nástrojů. Architektura pouze s dekodérem navíc vykazuje vynikající vlastnosti při škálování parametrů a efektivní správě KV cache v dlouhých kontextech.

2.3.2 Tokeny, tokenizér a embedding

Text, se kterým člověk pracuje ve formě slov a vět, není pro neuronovou síť přímo srozumitelný. Model interně operuje pouze s čísly a maticovými operacemi. Aby bylo možné přirozený jazyk zpracovat, musí projít procesem tokenizace a následného převodu na vektorové reprezentace.

Základní jednotkou, kterou model vnímá, je **token**. Token nemusí odpovídat celému slovu ani jednotlivému znaku — moderní jazykové modely využívají tzv. sub-word tokenizaci (nejčastěji algoritmy jako Byte-Pair Encoding či WordPiece). Běžná slova jsou často reprezentována jediným tokenem, zatímco méně častá slova, odborné výrazy nebo slova v jazycích s bohatou morfologií a diakritikou (jako je čeština) jsou rozložena do více dílčích tokenů. Průměrně jeden token v angličtině odpovídá přibližně 3 až 4 znakům. V češtině je spotřeba tokenů na slovo znatelně vyšší, což má přímý dopad na efektivní kapacitu kontextového okna i výpočetní náklady inference.

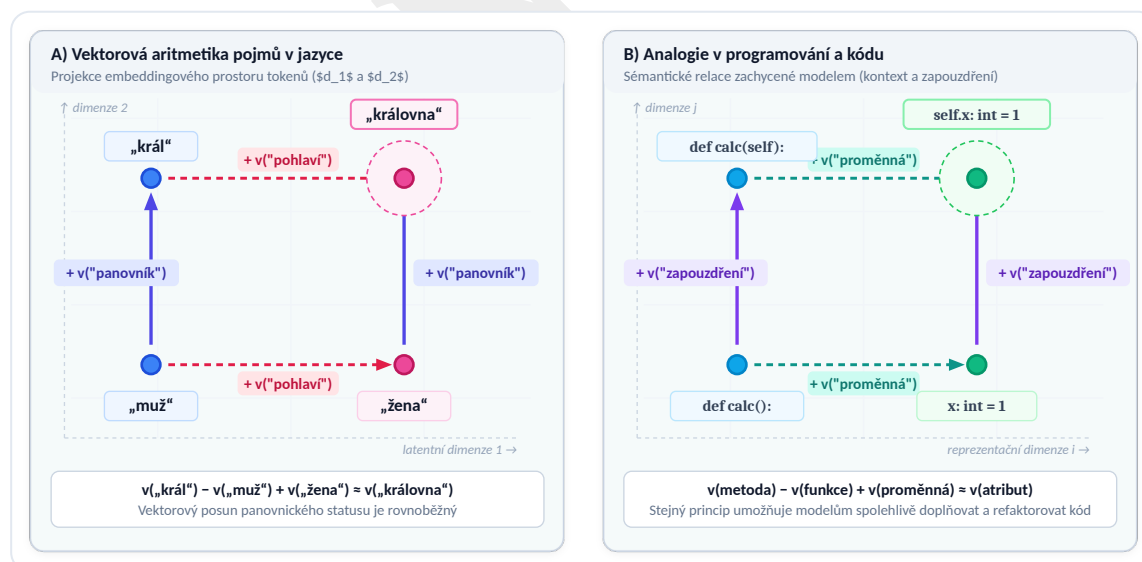
Převod mezi surovým textovým řetězcem a posloupností celočíselných identifikátorů (token IDs) zajišťuje **tokenizér**. Jedná se o deterministický program, který vychází z pevně daného slovníku (tzv. vocabulary, obvykle čítajícího 32 000 až 128 000 unikátních tokenů). Tokenizér provádí obousměrný proces: při vstupu rozseká text na tokeny a přiřadí jim číselné

indexy, při výstupu naopak generované indexy skládá zpět do souvislého textu (tzv. detokenizace).

Samotné číslo tokenu však nenese žádnou informaci o jeho významu. Proto následuje vrstva zvaná **embedding** (vektorové vnoření). Každý token je převeden na spojitý vícerozměrný vektor (typicky o dimenzi 4 096 či 8 192 čísel). V tomto geometrickém prostoru jsou slova s podobným významem či kontextem umístěna blízko u sebe (např. měreno kosinovou podobností).

Jednotlivé geometrické směry a posuny v prostoru navíc odpovídají konkrétním sémantickým relacím a abstraktním vlastnostem. Zjednodušeným a klasickým příkladem fungování tohoto prostoru je vektorová aritmetika pojmů: pokud vezmeme vektor reprezentující pojem „žena“ a přičteme k němu vektor reprezentující posun k „panovnickému stavu / královskému statusu“, výsledný bod v prostoru leží nejbližše reprezentaci slova „královna“ ($\vec{v}(\text{žena}) + \vec{v}(\text{královský status}) \approx \vec{v}(\text{královna})$). Model tak nepracuje se slovy jako s izolovanými symboly, ale manipuluje se vztahy mezi nimi jako s algebraickými posuny ve vícerozměrném prostoru — což se v programování projevuje schopností zachytit vztahy mezi deklarací proměnné, jejím použitím a typovým kontextem.

Aby architektura transformeru zohlednila také pořadí tokenů v sekvenci, přičítá se k sémantickému embeddingu poziční kódování (positional encoding, dnes standardně rotační embedding RoPE). Teprve takto vzniklé vektory vstupují do mechanismu pozornosti k dalšímu výpočtu.



Obrázek 1: Geometrická reprezentace sémantických vztahů v embeddingovém prostoru: A) Klasická vektorová aritmetika pojmů ($\vec{v}(\text{král}) - \vec{v}(\text{muž}) + \vec{v}(\text{žena}) \approx \vec{v}(\text{královna})$), B) Izomorfní algebraické relace v programování (funkce $\rightarrow$ metoda, proměnná $\rightarrow$ atribut).

Grafické znázornění na Obrázek 1 ilustruje, jak vícerozměrné vektorové vnoření zachycuje abstraktní sémantické relace. Na levém panelu (A) je patrné, že vektorový posun reprezen-

tující přechod k panovnickému stavu ($\vec{v}(\text{panovník})$) má téměř identický směr a velikost jak mezi „mužem“ a „králem“, tak mezi „ženou“ a „královnou“. Pravý panel (B) ukazuje analogický princip v programovacím kódu: model vnímá vztah mezi volně stojící funkcí a metodou zapouzdřenou ve třídě jako paralelní posun ve vektorovém prostoru k relaci mezi globální proměnnou a atributem objektu. Díky této prostorové struktuře dokáže LLM provádět konzistentní refaktoring a typovou inferenci.

Praktický dopad sub-word tokenizace na efektivitu a kapacitní limity agenta demonstruje Tabulka 1. Identické větné sdělení vyžaduje v češtině 33 tokenů oproti pouhým 12 tokenům v angličtině (měřeno tokenizérem `cl100k_base`). Český text tak spotřebovává 2,75× více prostoru v kontextovém okně a úměrně tomu zvyšuje finanční náklady i latenci inference. V autonomních vývojových pipeline je proto optimální vést vnitřní systémové prompty, technické plány a strukturované logy v angličtině, zatímco lidská interakce v zadání může probíhat v mateřském jazyce vývojáře.

Jazyk	Znění věty	Znaky	Tokeny
Angličtina	An autonomous software development system analyzes requirements and proposes code changes.	90	12
Čeština	Autonomní systém pro vývoj softwaru analyzuje požadavky a navrhuje změny kódu.	79	33

Tabulka 1: Empirické porovnání tokenové náročnosti ekvivalentního větného významu v angličtině a češtině (tokenizér `cl100k_base`).

2.3.3 Mechanismus pozornosti a jeho typy

Zatímco tokenizér a embedding transformují diskrétní text do spojitého vektorového prostoru, funkčním jádrem architektury transformeru je mechanismus pozornosti (*attention mechanism*). Pozornost umožňuje modelu dynamicky vyhodnocovat sémantické relace mezi jednotlivými tokeny v sekvenci bez ohledu na jejich vzájemnou vzdálenost, a překonat tak fundamentální limit starších rekurentních architektur (RNN a LSTM), které trpěly postupnou ztrátou kontextu.

2.3.3.1 Matematická podstata: Scaled Dot-Product Attention

Základní stavební jednotkou představenou v práci (3) je skalovaný skalární součin (*Scaled Dot-Product Attention*). Pro každý vstupní vektor tokenu se lineární projekcí generuje trojice vektorů:

- **Dotaz (Q – Query):** Vektor reprezentující informaci, kterou aktuální token v kontextu vyhledává.
- **Klíč (K – Key):** Vektor nesoucí charakteristiku a profil tokenu, podle něhož je identifikován.

- **Hodnota (V – Value):** Vektor obsahující vlastní sémantický obsah, který je v případě shody předán do dalších vrstev.

Míra relevance mezi libovolnou dvojicí tokenů je dána skalárním součinem QK^T . Výsledná matice afinity je škálována odmocninou dimenze klíče $\sqrt{d_k}$, což zabraňuje prudkému růstu číselných hodnot a následné saturaci funkce softmax (která by vedla k vymizení gradientů při trénování). Výstupní kontextová reprezentace je definována jako vážený součet hodnot V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Z matematické formulace vyplývá klíčová vlastnost: výpočet součinu QK^T pro sekvenci délky N vyžaduje sestavení matice o rozměrech $N \times N$. Výpočetní složitost i paměťové nároky proto rostou **kvadraticky** $O(N^2)$ vzhledem k délce kontextu, což při zpracování rozsáhlých repozitářů a dlouhých agentních historií představuje primární výkonnostní limit.

2.3.3.2 Směrové a funkční typy pozornosti

Podle původu vektorů Q , K a V a směru toku informací se rozlišují tři základní funkční typy:

1. **Vlastní pozornost (Self-Attention):** Vektory Q , K i V jsou odvozeny ze stejné vstupní sekvence. Každý token v kontextu tak přímo interaguje se všemi ostatními tokeny kódové báze (např. provázání volání funkce s její definicí v jiném modulu).
2. **Kauzální (maskovaná) pozornost (Causal / Masked Attention):** Klíčový princip autoregresivních modelů (*Decoder-only*). Aby model při predikci kódu nemohl „nahlížet do budoucna“, je na matici skalárních součinů před aplikací softmaxu uvalena kauzální maska ($M_{i,j} = -\infty$ pro $j > i$). Tím je matematicky zaručeno, že token na pozici i smí reflektovat výhradně předcházející tokeny na pozicích $j \leq i$.
3. **Křížová pozornost (Cross-Attention):** Dotazy Q pocházejí z generujícího dekodéru, zatímco klíče K a hodnoty V jsou přebírány z externí reprezentace — např. z vizuálního enkodéru zpracovávajícího screenshoty či diagramy v multimodálních architekturách.

2.3.3.3 Architektury organizace hlav (Head Architectures)

V praxi transformer nepočítá pozornost pouze jednou, nýbrž v paralelních projekčních podprostorech zvaných **hlavy** (*Multi-Head Attention* – MHA). Během inference se však ukázalo, že ukládání mezistavů klíčů a hodnot do mezipaměti grafické karty (*KV Cache*) pro všechny hlavy a vrstvy představuje kritické paměťové úzké hrdlo. To vedlo k vývoji několika architektonických variant (srovnání uvádí Tabulka 2):

1. **Multi-Head Attention (MHA)**: Původní architektura ((3)). Každá z h hlav má vlastní nezávislé projekční váhy pro Q , K i V . Poskytuje maximální reprezentační kapacitu, avšak velikost KV cache v paměti GPU roste strmě s počtem hlav a délkou kontextu.
2. **Multi-Query Attention (MQA)**: Architektura navržená Shazeerem (2019). Všechny dotazové hlavy sdílejí jedinou společnou hlavu klíčů a hodnot (K a V). Velikost KV cache v paměti klesá h -násobně, což radikálně zrychluje generování tokenů a snižuje paměťové nároky, avšak za cenu mírné degradace kvality u úloh vyžadujících komplexní logické uvažování.
3. **Grouped-Query Attention (GQA)**: Zlatý standard současných produkčních modelů (např. LLaMA 3 či Mistral; (4)). Dotazové hlavy jsou rozděleny do g skupin, přičemž každá skupina sdílí vlastní pár hlav K a V . Představuje optimální kompromis: dosahuje kvality srovnatelné s MHA při paměťové úspoře blízké MQA (obvykle 12,5–25 % původní velikosti KV cache).
4. **Multi-head Latent Attention (MLA)**: Moderní architektura vyvinutá pro modely DeepSeek-V2/V3. Místo prostého sdílení hlav komprimuje vektory klíčů a hodnot do nízkorozměrného latentního prostoru pomocí společné maticové komprese (*low-rank joint compression*). Během inference se do KV cache ukládá pouze kompaktní latentní vektor, což dramaticky redukuje paměťovou stopu a umožňuje obsluhu masivních kontextových oken.

Typ pozornosti	Hlavy Q	Hlavy K, V	Velikost KV Cache	Charakteristika a využití
MHA	h	h	100 % (báze)	Původní Transformer, GPT-3. Maximální exprese, vysoké nároky na VRAM.
MQA	h	1	$\frac{1}{h}$ (3–6 %)	Falcon, PaLM. Extrémní úspora paměti, možný mírný pokles přesnosti.
GQA	h	g ($1 < g < h$)	$\frac{g}{h}$ (12–25 %)	LLaMA 3, Mistral. Špičkový standard kombinující přesnost a propustnost.
MLA	h	Latentní komprese	5–10 %	DeepSeek-V2, DeepSeek-V3. Nízkorozměrná latentní projekce s vysokou věrností.

Tabulka 2: Srovnání architektury organizace hlav mechanismu pozornosti a jejich dopad na paměťovou náročnost KV cache.

2.3.3.4 Hardwarové a algoritmické optimalizace pro škálování kontextu

Kvadratická závislost $O(N^2)$ si vyžádala vývoj algoritmických a nízkoúrovňových hardwarových akcelerací:

- **Sliding Window a Sparse Attention (SWA):** Pozornost je omezena na lokální okno W nejblížešších sousedních tokenů (např. $W = 4096$ v modelu Mistral). Ačkoli v jedné vrstvě model nevidí celou sekvenci, skládáním vrstev nad sebou efektivní kontextové pole roste, zatímco výpočetní složitost klesá na lineární $O(N \times W)$.
- **FlashAttention:** Převratná optimalizace navržená Dao et al. ((5), následovaná architekturami FlashAttention-2 a FlashAttention-3). FlashAttention neprovádí žádnou ztrátovou

aproximaci — matematický výsledek je exaktní. Podstata spočívá v hardwarovém uvědomění (*IO-awareness*): namísto ukládání ohromné mezilehlé matice pozornosti $N \times N$ do pomalé globální paměti grafické karty (HBM) rozděluje výpočet na bloky (tzv. *tiling*) a provádí normalizaci online algoritmem softmaxu přímo v rychlé čipové mezipaměti SRAM procesoru GPU. Tím dramaticky redukuje paměťové přenosy a umožňuje praktické škálování kontextového okna na stovky tisíc až miliony tokenů.

2.3.4 Multimodální modely

Architekturu transformeru lze aplikovat i mimo oblast zpracování přirozeného jazyka. V moderní praxi lze tokenizovat v podstatě jakýkoli diskrétní či spojitý signál — ať už jde o rastrový obraz, sekvenci snímků videa, zvukové vlny nebo trajektorie pohybů v robotice. Zásadní posun nastává ve chvíli, kdy model disponuje oddělenými projekčními vrstvami pro různé typy vstupů (např. kombinací textového tokenizéru a vizuálního enkodéru, jako je ViT — *Vision Transformer*) a mechanismus pozornosti (*cross-attention*) operuje společně nad tokeny textu i obrazu. Model tak dokáže propojovat vizuální a textové sémantické reprezentace ve sdíleném vektorovém prostoru.

~~V kontextu automatizovaného vývoje softwaru a autonomních pipeline (jako je systém DarkFactory) se multimodální schopnosti uplatňují ve třech klíčových oblastech:~~

- ~~1. **Vizuální regresní testování:** Model dokáže porovnat referenční snímek uživatelského rozhraní se stavem vygenerovaným v CI (např. při testování webových komponent bez hlavním prohlížečem) a identifikovat nežádoucí posuny rozvržení či stylové chyby.~~
- ~~2. **Diagnostika chyb z artefaktů CI:** Při selhání integračních testů může pipeline předat agentovi screenshot chybové obrazovky nebo interaktivního prvku, z něhož agent rozpozná příčinu selhání snáze než z pouhého textového stack trace.~~
- ~~3. **Interpretace grafických zadání:** Vývojář může v zadání úkolu (GitHub Issue) specifikovat požadovanou změnu formou náčrtku, wireframu či diagramu komponent. Multimodální agent takový grafický podklad přímo zanalyzuje a převede jej na strukturovaný kód.~~

Vizuálně schopné modely (tzv. *Vision-Language Models* neboli VLM) tak nestaví na principiálně odlišných teoretických základech, nýbrž sdílejí identickou architekturu dekodérového transformeru jako čistě textové modely: rastrový obraz je rozložen na pravidelnou mřížku dlaždic (*patches*), které jsou lineární projekcí transformovány na vizuální tokeny se stejnou dimenzí vnoření jako tokeny textové. Ačkoli v kontextu softwarového inženýrství nabízejí tyto modely teoretický potenciál pro doplňkové úlohy (např. interpretaci grafických wireframů v zadání, vizuální regresní testování webových komponent či analýzu screenshotů z chybujících běhů v CI), v současné praxi zůstávají autonomní kódovací pipeline primárně postaveny na textových reprezentacích — tedy na práci se zdrojovým kódem,

abstraktními syntaktickými stromy (AST), unifikovanými diffy v Gitu a textovými protokoly testovacích nástrojů.

2.3.5 Context

2.3.5.1 Turn

Každému diskrétnímu kroku ve výměně informací mezi okolním prostředím a jazykovým modelem se v agentních architekturách říká tah (*turn*). Na rozdíl od jednoduchého konverzačního rozhraní, kde dochází pouze ke střídání uživatele a asistenta, agentní smyčka (typicky implementující vzor ReAct (6)) rozlišuje tři základní typy tahů:

1. **Tah uživatele či prostředí (*User / Environment Turn*):** Vnáší do kontextu nové zadání, externí událost (např. spuštění GitHub webhooku) nebo doplňující instrukce.
2. **Tah modelu (*Model / Assistant Turn*):** Reprezentuje jedno spuštění inference neuro-nové sítě. Model na základě dosavadní historie emituje buď finální textovou odpověď, nebo strukturovaný požadavek na vyvolání nástroje (*tool call*).
3. **Tah vykonání nástroje (*Tool Execution Turn*):** Běhové prostředí (harness) provede požadovanou operaci — např. spuštění skriptu v terminálu či čtení souboru — a její výsledek vloží do kontextu jako syntetický tah určený pro navazující uvažování modelu. Striktní oddělení těchto fází a jejich deterministická serializace do historie kontextu mají zásadní dopad na výpočetní efektivitu: umožňují inferenčnímu serveru plně využívat cachování klíčů a hodnot (*KV cache*), neboť neměnná historie předchozích tahů nemusí být při každém kroku znovu přepočítávána.

2.3.5.2 Context Window

Model je schopen přijmout pouze omezený objem vstupu; tomuto limitu se říká kontextové okno (*Context Window*) a vyjadřuje se v počtu tokenů. Na rozdíl od slovníku (*vocabulary*), který vymezuje pouze repertoár známých tokenů, je maximální délka kontextového okna určena architekturou pozičního kódování (např. škálováním frekvenčních bází v RoPE) a hardwarovou náročností mechanismu pozornosti — kvadratickou složitostí $O(N^2)$ a velikostí alokované KV cache v operační či grafické paměti.

2.3.5.3 Compaction

Představuje proces sumarizace a zkrácení historie, který řídící harness iniciuje ve chvíli, kdy zaplnění kontextového okna dosáhne stanoveného prahu. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí pro bezztrátovou syntézu dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí k syntéze dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Výsledný zkrácený kontext následně v kontextovém okně nahradí původní rozsáhlou historii kroků.

2.3.5.3.1 Limity rekurzivní komprese a sémantický posun (Semantic Drift)

Ačkoliv je proces komprese historie v řadě základních implementací prezentován jako přímočaré řešení konečné kapacity kontextového okna, z hlediska teorie informace a systémového inženýrství představuje **destruktivní ztrátovou kompresi**. Při zkracování dosavadní trajektorie model trpí výrazným **konfirmačním zkreslením** (*confirmation bias*): autoregresivní model při generování souhrnu nevybírá fakta nestranně, nýbrž sumarizuje primárně ty aspekty historie, které sám považuje za relevantní vzhledem ke svým vnitřním statistickým asociacím.

Tato ztrátovost má fatální dopad na integritu vývojového kontextu:

1. **Vymazání deterministických detailů:** Zkrácením nevratně zanikají přesná čísla řádků, signatury privátních funkcí, plné cesty souborů a doslovná znění chybových hlášení kompilátoru či testovacího frameworku (*stack traces*).
2. **Ztráta negativních omezujících podmínek:** Zvláště náchylné k vymizení jsou explicitní negativní mantinely stanovené uživatelem v úvodu úlohy (např. „neměň veřejné rozhraní modulu X“ nebo „nepřidávej žádné externí závislosti“). V syntetizovaném souhrnu bývají tyto kritické podmínky zobecněny či zcela opomenuty, což v navazujících tazích vede k jejich okamžitému porušení.
3. **Sémantický posun (Semantic Drift):** Při dlouhotrvajících komplexních úlohách vyžadujících vícenásobnou rekurzivní kompresi dochází ke kaskádovému kumulování chyb (analogii hry na tichou poštu). Pokud je stav S_{k+1} generován jako souhrn předchozího souhrnu S_k a nového přírůstku Δ_k , drobné aproximační odchylky či halucinace z kola k jsou v kole $k + 1$ přijaty jako nezvratná historická fakta. Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem repozitáře a původním zadáním.

K překonání fundamentálních limitů textové rekurzivní komprese se v moderních agentních architekturách uplatňují tři rigorózní alternativy:

- **Hierarchická epizodická paměť a vyhledávání (RAG):** Kompletní doslovná historie tahů, výpisů nástrojů a diffů se nezkracuje přímo v kontextu, nýbrž persistuje do externí databáze (např. fulltextového či vektorového indexu). Do aktivního kontextu jsou následně deterministicky či sémanticky injektovány pouze ty minulé fragmenty, které jsou bezprostředně relevantní pro řešený podúkol.
- **Persistentní graf stavu projektu (Project State Graph):** Namísto nestrukturovaného volného textu udržuje harness explicitní strukturovaný stav projektu (např. formou formálního stavového schématu či grafového modelu), který deterministicky eviduje modifikované soubory, seznam nezodpovězených otázek, stav integračních testů a nez-

měnitelné invarianty. Aktualizace tohoto stavu se řídí přísným schématem a nepodléhá volné narativní degradaci.

- **Selektivní prořezávání KV cache (Selective KV Cache Eviction):** Místo manipulace s textovými tokeny probíhá řízení paměti přímo na úrovni tenzorů v inferenčním enginu. Algoritmy jako StreamingLLM (7) či H_2O (Heavy Hitter Oracle) (8) identifikují a v paměti trvale uchovávají tzv. kotevní tokeny pozornosti (*attention sinks*) a matematicky nejvýznamnější minulé stavy, zatímco redundantní mezilehlé stavy bezpečně uvolňují bez nutnosti destruktivního přepisování promptu.

2.3.5.4 Context Rot a evaluace vybavování (Needle In A Haystack)

Označuje empiricky zdokumentovanou degradaci schopnosti modelu věnovat rovnoměrnou pozornost všem částem historie (tzv. jev *Lost in the Middle* (9)). Čím plnější je kontextové okno, tím méně jsou modely schopny spolehlivě vybavovat jemné detaily z úvodu sezení a dodržovat negativní omezující podmínky zadání.

Ke standardizovanému testování schopnosti modelu přesně vybavovat informace z rozsáhlého kontextového okna slouží syntetický evaluační test **Needle In A Haystack** (NIAH – „jehla v kupce sena“). Princip spočívá ve vložení izolovaného, na kontextu nezávislého faktu (např. unikátního klíče či specifického nastavení konfigurace) do různé relativní hloubky (od 0 % do 100 %) dlouhého distraktoru textu (čítajícího desítky tisíc až miliony tokenů). Model je následně vyzván k zodpovězení dotazu, jehož vyřešení závisí výhradně na této vložené informaci.

Zatímco moderní frontier modely dosahují na jednoduchém syntetickém testu NIAH téměř stoprocentní úspěšnosti napříč celou délkou okna, v reálném softwarovém vývoji vyvstává zásadní omezení: práce v repozitáři vyžaduje tzv. **Multi-Needle Reasoning** — schopnost současně nalézt, propojit a logicky syntetizovat několik vzájemně závislých informací (např. signaturu funkce v jednom modulu, její volání v druhém a konfigurační flag ve třetím). V takovém scénáři se naplno projevuje degradace pozornosti: s rostoucím objemem historie v kontextovém okně rapidně klesá spolehlivost křížového uvažování, což v dlouhých agentních sezeních vede k přehlédnutí okrajových podmínek a regresi v kódu.

2.3.5.5 KV Caching

KV caching slouží ke snížení výpočetní náročnosti modelu při generování tokenů. Místo toho, abychom při každém kroku inference znovu od začátku počítali pozornost pro celou dosavadní konverzaci, ponechává inferenční engine v paměti uložené již vypočtené vektory klíčů a hodnot (*Key-Value pairs*) a v každém kroku počítá a přidává pouze nově vygenerovaný token.

2.3.6 Prompt a kontextové inženýrství

Prompt engineering (inženýrství promptů) je disciplína zaměřená na systematický návrh, formulaci a optimalizaci textových instrukcí předkládaných modelu (10). V autonomních agentních systémech nepředstavuje prompt pouhou volnou konverzací, ale slouží jako závazný kontrakt vymezující chování, práva a bezpečnostní mantinely agenta. Mezi klíčové techniky patří:

- **Systémový prompt (System Prompt):** Základní direktiva definující identitu agenta, dostupné nástroje a striktní provozní pravidla (např. pravidla pro zachování neměnnosti existujících testů, konvence pro formát commitů či zákaz destruktivních operací v repozitáři).
- **Few-shot prompting:** Technika vložení několika vzorových příkladů (vstup-výstup) přímo do kontextu. Tím model získá konkrétní představu o požadovaném schématu (např. syntaxi konfiguračního souboru `darkfactory.json`) bez nutnosti dodatečného dotrénování parametrů.
- **Chain-of-Thought (myšlenkový řetězec):** Vedení modelu k explicitní formulaci mezikroků a vnitřní dedukce před vygenerováním konečného kódu či akce. Rozklad komplexního zadání na postupné logické kroky zásadně potlačuje halucinace a tvoří základ fáze rozvahy (*Thought*) v cyklu ReAct.
- **Injekce dynamického kontextu:** Průběžné doplňování promptu o aktuální stav repozitáře, stromovou strukturu souborů, chybové výpisy kompilátoru a výsledky testů, díky čemuž agent operuje nad reálnými fakty namísto odhadů.

2.3.6.1 Úskalí negativních instrukcí a princip afirmativního vymezení

Zvláštní výzvou při formulaci systémových pravidel a zadání úkolů je definice omezujících podmínek pomocí zákazů a negativních instrukcí (např. „nikdy nemaž existující testy“, „neupravuj soubory mimo složku src“). V praxi je empiricky prokázáno, že jazykové modely negativní příkazy často porušují nebo zcela ignorují.

Tento jev má hluboké opodstatnění v samotném matematickém fungování mechanismu pozornosti transformeru:

1. **Pozornost zaměřená na přítomná slova:** Matice pozornosti (QK^T) vyhodnocuje sémantickou relevanci na základě **přítomných** tokenů. Zmínka zakázané operace (např. `test_suite.py` ve větě „Za žádných okolností nemaž test_suite.py“) aktivuje v embeddingovém prostoru silné vektorové asociace k danému souboru i k operaci smazání.
2. **Asymetrie negace v autoregresivní predikci:** Negace představuje logický operátor vysokého řádu vyžadující složenou kompozici významu. Při autoregresivním generování token po tokenu však model často podlehně silnější statistické asociaci mezi přítomnými sémantickými pojmy dříve, než uplatní logický kontext záporové částice.

V robustním inženýrství agentních systémů se proto uplatňují dva komplementární principy:

- **Afirmativní formulace pravidel:** Pravidla se namísto zákazů formulují pozitivním vymezením požadovaného chování a mantinelů (např. „Upravuj výhradně soubory deklarované v seznamu `target_files`“, „Před jakoukoli změnou ověř běh stávajících testů“).
- **Deterministická exekuční ochrana v harnessu:** Bezpečnostní mantinely a zákazy destruktivních operací nesmí záviset na poslušnosti jazykového modelu vůči promptu. Řídící harness systému DarkFactory proto prosazuje striktní omezení deterministicky na úrovni běhového prostředí — např. připojením chráněných cest v souborovém systému pouze pro čtení (*read-only mount*), sandboxováním exekuce terminálových příkazů a nezávislou validací změn v pull requestu před sloučením.

2.3.7 Agent vs Chatbot

Fundamentální rozdíl mezi chatbotem a agentem spočívá v rozsahu interakce s okolním světem: zatímco chatbot operuje výhradně v uzavřeném konverzačním rozhraní a generuje textové odpovědi, agentem se systém stává ve chvíli, kdy je vybaven sadou výkonných nástrojů (*tools*). Prostřednictvím nich dokáže aktivně zkoumat repozitář, vyhledávat informace na webu, modifikovat soubory a autonomně vykonávat příkazy v cílovém výpočetním prostředí.

2.3.8 Harness a System Prompt

Harness je řídicí program obklopující jazykový model. Odlišuje se od inferenčního jádra (*inference engine*), které provádí samotné maticové výpočty sítě: harness má na starost rozhraní mezi uživatelem a agentem či chatbotem, předávání systémového promptu, bezpečné spouštění nástrojů a řízení iterativní smyčky neboli ReAct smyčky. Mezi typické příklady patří webová aplikace ChatGPT, terminálové rozhraní Claude Code, desktopová aplikace Codex a další agentní prostředí.

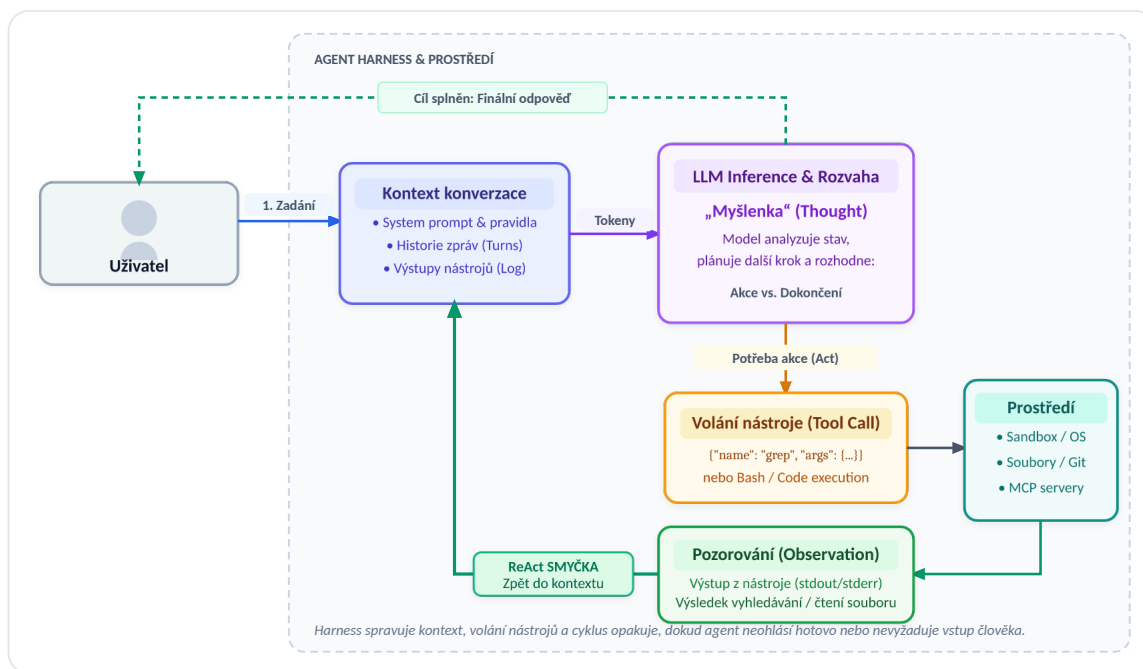
2.3.9 ReAct smyčka

Smyčka ReAct (*Reasoning and Acting*) tvoří základní stavební kámen, který z jazykového modelu vytváří autonomního agenta namísto pouhého pasivního generátoru textu (6). Princip nespočívá v jednorázovém vygenerování kódu či odpovědi, nýbrž v soustavné alternaci vnitřní rozvahy (*Reasoning* neboli *Thought*) a vnějšího jednání (*Acting* neboli *Tool Call*). Tento princip byl formálně zaveden v práci *ReAct: Synergizing Reasoning and Acting in Language Models* ((6), [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)).

Jak znázorňuje diagram na Obrázek 2, po přijetí uživatelského zadání dochází k následujícímu iterativnímu cyklu:

1. **Fáze rozvahy (*Thought*):** Model v rámci inference analyzuje aktuální stav kontextu a formuluje hypotézu či myšlenkový postup, na jehož základě určí nejbližší nezbytný krok.

2. **Vyvolání akce (*Tool Call*):** Pokud krok vyžaduje externí operaci, model emituje strukturovaný požadavek na volání nástroje.
3. **Vykonání v harnessu a pozorování (*Observation*):** Řídící harness zachytí požadavek, bezpečně jej provede v cílovém prostředí (terminál, souborový systém či MCP server) a vrácený výstup připojí do kontextového okna jako nové pozorování.
4. **Navazující iterace:** Aktualizovaný kontext je předložen modelu v dalším tahu, čímž agent adaptivně reaguje na reálnou odezvu prostředí namísto slepé generace.



Obrázek 2: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.9.1 Řízení divergence, detekce uvíznutí a rozpočet tahů (Step Budget)

Základní teoretický popis smyčky ReAct předpokládá, že proces přirozeně konverguje k vyřešení problému nebo k rozhodnutí modelu o předání finální odpovědi uživateli. V reálné autonomní softwarové praxi je však tento předpoklad zásadně lichý: jazykový model je stochastický systém bez inherentních garancí terminace a při řešení neúplně specifikovaných či složitých chyb v repozitáři často podléhá patologickým stavům divergence:

1. **Perseverance a zacyklení (*Perseveration & Infinite Loops*):** Model opakovaně emituje identické volání nástroje se zcela shodnými parametry (např. čte tentýž neexistující soubor, provádí tentýž neúspěšný regex match nebo spouští tentýž selhávající test), a to i přesto, že z prostředí obdržel chybovou odezvu (*Observation*). Z hlediska pravděpodobnostní dynamiky dochází ke vzniku tzv. **atraktoru v kontextu**: přítomnost chybného pokusu a chybové hlášky v bezprostřední historii paradoxně zvyšuje podmíněnou pravděpodobnost, že model v dalším tahu vygeneruje velmi podobný tokenový sled.

2. **Oscilace a tĕkání (*Thrashing*)**: Model střídavě přepíná mezi dvěma protichůdnými zásahy (např. úprava funkce A vyvolá regresi v modulu B, model následně zedituje modul B a rozbije modul A, načež změny cyklicky invertuje bez hlubšího pochopení systémové příčiny).

3. **Nekontrolovaná spotřeba zdrojů (*Context Runaway*)**: Bez striktního vnějšího omezení může divergentní smyčka v krátkém čase zcela zaplnit kontextové okno a vyčerpat finanční rozpočet na API volání.

Z těchto důvodů nesmí být rozhodnutí o ukončení cyklu ponecháno výhradně na autonomní vůli modelu. Řídící harness musí vůči modelu vystupovat jako deterministický dozorčí mechanismus (*watchdog*) a prosazovat následující vrstvy ochrany:

- **Rozpočet tahů a nákladů (*Step & Cost Budget*)**: Každému sezení či dílčímu podúkolu je přiřazen pevný rozpočet maximálního počtu tahů $T_{\max}$ (typicky 25–50 kroků) a finanční strop pro spotřebu tokenů. Překročení tohoto rozpočtu vede k okamžitému a bezpodmínečnému zastavení inference.

- **Algoritmická detekce uvíznutí (*Stuck Detection*)**: Běhové prostředí počítá pro každé volání nástroje kanonický hash jeho identifikátoru a normalizovaných argumentů:

$$h_t = \text{hash}(\text{nástroj}, \text{canonicalize}(\text{argumenty}))$$

Harness v klouzavém okně posledních k tahů (např. $k = 3$) detekuje identické volání $h_t = h_{t-1} = \dots = h_{t-k+1}$ nebo periodické vzorce v posloupnosti hashů.

- **Dvoustupňová intervenční strategie**:

1. **Měkká intervence (*Synthetic Warning*)**: Při první detekci repetice harness přeruší standardní tok a injektuje do kontextu syntetickou systémovou observaci s vysokou prioritou (např. „Varování harnessu: Nástroj byl opakovaně vyvolán s identickými argumenty bez pozitivního posunu. Přehodnotte hypotézu, změňte diagnostický přístup nebo požádejte o doplňující instrukce.“). Tím se účinně rozbije pravděpodobnostní atraktor v matici pozornosti.

2. **Tvrdá intervence (*Circuit Breaker*)**: Pokud perseverace pokračuje i po varování, harness cyklus okamžitě ukončí, provede deterministický návrat (*rollback*) neuložených změn v repozitáři na poslední stabilní gitový commit a eskaluje stav lidskému operátorovi jako neřešitelnou anomálii.

2.3.10 Vyvolávání nástrojů

Abychom z jazykového modelu vytvořili autonomního agenta, musíme jej vybavit rozhraním pro interakci s okolním prostředím — tzv. nástroji (*tools*). V praxi existují dva převažující přístupy k realizaci nástrojů:

1. **Strukturované volání nástrojů (*Function / Tool Calling v JSON*)**: Model ve svém výstupu zanechá strukturované volání odpovídající schématu zadanému v definici ná-

stroje. Řídicí harness toto volání zachytí, vykoná požadovanou funkci a vrátí výsledek zpět agentovi opět jako strukturovaná data. Tato možnost je spolehlivá pro jednoúčelové operace (např. integraci do podnikových systémů či zákaznické podpory — angl. *customer support*). Pro komplexní softwarový vývoj však představuje nevýhodu vysoká režie formátu JSON: velké množství formátovacích znaků se převádí na tokeny, což zbytečně plní kontextové okno a může vést k degradaci pozornosti (*Context Rot*).

2. **Přímé spouštění kódu (Code Execution):** Druhou možností je poskytnout agentovi plnohodnotné běhové prostředí (např. Bash, Python či TypeScript). Agent vygeneruje kód pro splnění svého záměru a harness jej spustí buď po jednotlivých příkazech, nebo jako ucelený skript. Z bezpečnostních důvodů je nezbytné, aby agent nepracoval přímo na nechráněném hostitelském počítači, nýbrž ve virtuálním prostředí odděleném bezpečnostní vrstvou — tzv. sandboxu (pískovišti). Tento způsob je pro softwarový vývoj nejefektivnější.

 **Hlubková kritika / Oponentura:** Iluzorní bezpečnost pískoviště: Druhý přístup (přímé spouštění kódu v kontejneru) práce nekriticky prezentuje jako „nejefektivnější“, zamlčuje však gigantická bezpečnostní a operační rizika. Běžný Docker kontejner není plnohodnotná bezpečnostní hranice (*security boundary*). Spuštění netestovaného kódu generovaného modelem s přístupem k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky tajných environmentálních proměnných (GitHub tokeny, API klíče k LLM poskytovatelům) přes postranní síťové kanály a nevratné poškození repozitáře. Práce postrádá jakoukoli analýzu formální izolace (např. microVM architektury jako AWS Firecracker, gVisor či striktní network namespaces) a nezodpovídá otázku, co zabráni kompromitovanému modelu zneužít CI runner jako odrazový můstek pro kompromitaci infrastruktury.

2.3.11 Dovednosti, skripty a záchytné body

Tyto standardy vznikly proto, aby vývojáři mohli modulárně upravovat chování agenta a rozšiřovat jeho schopnosti pro specifické doménové úlohy:

- **Dovednost (Skill):** Samostatný adresář sdružující instrukce, reference a pomocné soubory. Klíčovým prvkem je soubor `SKILL.md`, který využívá hlavičku v metadatovém formátu YAML frontmatter (ohrazenou trojicí pomlček `---`). V hlavičce je definován název a stručný popis dovednosti. Řídicí harness do základního systémového promptu vkládá pouze tato metadata; samotný text podrobného návodu se do kontextu načte až ve chvíli, kdy agent danou dovednost vyvolá. Tím se efektivně šetří kapacita kontextového okna.
- **Skript (Script):** Jednoúčelový spustitelný program (např. v jazyce Python či Bash). Skripty jsou pro efektivitu práce vitální: agent nemusí generovat každý příkaz z paměti, ale spouští deterministické a otestované postupy pro analýzu a transformaci kódu.

- **Záchytný bod (Hook):** Skript, který se v harnessu automaticky vyvolá při určité systémové události či stavovém přechodu (např. při inicializaci sezení, před odesláním promptu modelu nebo při selhání nástroje). Hooky umožňují vynucovat bezpečnostní pravidla a logování nezávisle na vůli samotného modelu.

2.3.12 MCP Server

Model Context Protocol vznikl v laboratořích společnosti Anthropic jako standardizovaný způsob pro efektivní interakci agentů s API servery. Ve své podstatě se jedná o nástroje postavené na rozhraní externího API, tudíž protokol sjednocuje různé standardy do jediného otevřeného rozhraní a přidává kontext — metadata pro každou přítomnou metodu, aby agent předem věděl, co od ní očekávat, a nemusel sám odhadovat fungování backendu. Praktický význam specifikace Model Context Protocol (11) spočívá ve sjednocení dříve fragmentovaného ekosystému proprietárních rozhraní a jednoúčelových integračních skriptů. Řídící harness se díky standardu stává univerzálním klientem a veškeré externí schopnosti, datové zdroje či nástroje se integrují jako samostatné procesy — tzv. MCP servery. Komunikace probíhá přes standardizovaný protokol JSON-RPC, a to buď lokálně prostřednictvím standardního vstupu a výstupu (`stdio`), nebo vzdáleně pomocí protokolu HTTP se Server-Sent Events (`SSE`). Tento modulární přístup striktně odděluje běhové prostředí agenta od samotné implementace nástrojů: libovolnou schopnost (např. přístup k databázi, vyhledávání v repozitáři nebo správu GitHub úkolů) stačí naimplementovat jednou a lze ji okamžitě zpřístupnit jakémukoli kompatibilnímu agentovi bez nutnosti zásahu do kódu samotného harnessu.

2.3.13 Meta Harness

V reálném světě se ukázalo, že nejefektivnější harness je takový, který dá samotnému agentovi rozhodovací pravomoc nad vlastní architekturou a umožňuje kontinuální změnu, tedy samořízenou evoluci (12). Tento přístup je mimořádně efektivní pro maximalizování pracovní kapacity agenta, v praxi se však hodí především pro osobního agenta. U komerčních nasazení, jako je zákaznická podpora (*customer support*), provozovatel naopak striktně vyžaduje deterministické mantinely a vylučuje, aby model modifikoval své vlastní provozní instrukce.

2.3.14 Subagenti

Když je agent vybaven nástrojem umožňujícím vyvolat další specializovanou instanci jazykového modelu, zadat jí dílčí úlohu, asynchronně s ní komunikovat a sledovat její postup, efektivita řešení rozsáhlých softwarových problémů dramaticky roste. V teorii autonomních systémů se tyto delegované entity označují jako subagenti (*subagents*). Tato architektura umožňuje hierarchickou dekompozici problému: hlavní koordinační agent (*orchestrator*) udržuje globální strategii a pověřuje úzce profilované subagenty

izolovanými činnostmi — například řešerší dokumentace, prozkoumáním rozsáhlého adresářového stromu repozitáře nebo syntaktickou opravou konkrétního modulu. Zásadní architektonickou výhodou je ochrana a izolace kontextového okna: rozsáhlý a výpočetně náročný průzkumný kontext subagenta se po dokončení úkolu zahodí a rodičovskému orchestrátoru je předán pouze syntetizovaný, čistý výsledek. Tím se zabráňuje zahlcení primárního kontextu (*context pollution*) a degradaci kognitivních schopností hlavního agenta.

2.3.15 Workflows neboli Graph Engineering

Jakmile se snažíme organizovat nebo automatizovat úlohu složitější než několik přímočarých kroků, lineární sekvence promptů přestává stačit. V takovém okamžiku přicházejí na řadu pracovní postupy (*workflows*), v teoretické informatice reprezentované jako grafy. Grafy se v softwarovém vývoji používají v mnoha situacích. Graf se skládá z uzlů (*nodes*) a hran (*edges*) — v kontextu agentních systémů každý uzel představuje izolovaného agenta s připraveným systémovým promptem, dovednostmi a nástroji, zatímco spoje mezi nimi vymezují tok informací a posloupnost práce. Abstrakcí tohoto grafového systému jsme schopni tuto pipeline nasadit na libovolnou úlohu, a to i dynamicky za pomoci orchestrace nadřazeným agentem.

Tento koncept se v informatice označuje jako orientovaný acyklický graf (**DAG** — *Directed Acyclic Graph*). Uzly grafu představují samostatné, specializované výpočetní kroky či izolované agenty, zatímco orientované hrany určují pořadí závislostí a tok kontextových informací. V moderních orchestrátorech a CI/CD platformách (zejména v GitHub Actions) se závislosti mezi úlohami deklarují pomocí direktivy `needs: [...]`.

Jak podrobně rozvádí praktická část této práce na architektuře systému DarkFactory, celý životní cyklus automatizovaného požadavku je strukturován právě jako DAG složený z pěti klíčových fází:

1. **Detekce a kontext:** rozpoznání prostředí projektu, načtení konfiguračního souboru `darkfactory.json` a příslušných systémových pravidel.
2. **Interpretace a plánování:** formulace přesného technického postupu a vytvoření plánovacího úkolu dříve, než dojde k samotnému zásahu do kódu.
3. **Implementace (kódování):** spuštění agenta s přesně vymezenou sadou nástrojů v izolované větvi repozitáře.
4. **Verifikace a testování:** automatické sestavení projektu, spuštění testovací sady a kontrola kvality změn.
5. **Schvalovací brána (Governance):** podmíněné zastavení toku grafu a vyžádání lidské kontroly před finálním sloučením pull requestu.

Zásadní předností grafového uspořádání je determinismus a striktní bezpečnost: pokud kterýkoli uzel selže (např. automatizované testy detekují regresi), exekuce se v dané větvi

grafu okamžitě přeruší a navazující kroky se nespustí, což zabraňuje poškození hlavní vývojové linie.

2.4 Human in the loop neboli člověk ve smyčce

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem; cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné nebo kde chybí měřítko správnosti.

2.4.1 Schvalovací body

Schvalovací bod je místo, kde se proces zastaví a čeká na potvrzení. Jeho umístění je kompromisem: příliš mnoho schvalování popírá smysl automatizace, příliš málo znamená ztrátu kontroly. Osvědčeným řešením je schvalovat *záměr* (co se má udělat) a *plán* (jak se to má udělat) dříve, než vznikne kód.

2.4.2 Dohledatelnost

Aby byla automatizovaná změna přezkoumatelná, musí být zřejmé, z jakého požadavku vzešla. Uchování doslovného znění původního zadání je proto součástí návrhu, nikoli formalitou: parafráze ztrácí význam, který do zadání vložil ten, kdo je formuloval.

2.4.3 Eskalace a transparence selhání

Systém, který své vlastní selhání zamlčí nebo zamete pod koberec, je nebezpečnější než systém, který zjevně havaruje: nespolehlivost se v něm stává neviditelnou a postupně eroduje důvěru uživatele v celý autonomní provoz. V konceptu člověka ve smyčce proto hlášení chyb a eskalace selhání netvoří pouhý doplňkový technický detail, nýbrž fundamentální bezpečnostní pilíř. Pokud autonomní agent narazí na vyčerpání kontextového okna, syntaktickou chybu neřešitelnou v rámci rozpočtu tahů nebo selhání integračních testů, nesmí skončit tichým uváznutím či nekonečným cyklem. Místo toho musí harness deterministicky zachytit chybový stav, sestavit strukturovaný diagnostický protokol (obsahující chybový stack trace, diff provedených změn a stav kontextu) a srozumitelně jej eskalovat člověku formou notifikace či dedikovaného incidentu. Tím je zajištěna plná observabilita a okamžitá lidská dohledatelnost.

3 Praktická část

Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (13).

3.1 Cíl a rozsah systému

Systém má převzít rutinní kroky vývojového procesu: přijetí požadavku, jeho interpretaci, naplánování, provedení změny a její ověření. Nemá nahradit rozhodování o tom, co se má stavět; to zůstává člověku, a systém je navržen tak, aby si toto rozhodnutí vyžádal dříve, než začne pracovat.

3.2 Architektura

 **Strukturální upozornění: Architektura v rekonstrukci:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory byla dosavadní implementační dekompozice modulů a skriptů dočasně nahrazena zástupnými strukturálními bloky. Nová architektura, komponentní diagram a specifikace rozhraní budou doplněny po stabilizaci nové verze.

Systém DarkFactory je navržen jako modulární stavebnice složená z řídicích orchestračních modulů, šablon pracovních postupů pro kontinuální integraci a hermetického^{*} běhového prostředí. Architektura striktně odděluje deklarativní konfiguraci konkrétního repozitáře od samotné logiky orchestrace.

3.2.1 Koncepce a systémové vrstvy

 **Návrh na vylepšení:** Placeholder: Zde bude doplněna nová vrstevnatá dekompozice přestavěného systému DarkFactory včetně nového komponentního schématu nahrazujícího původní skriptové jádro.

3.2.2 Sdílení pracovních postupů

 **Návrh na vylepšení:** Placeholder: Zde bude popsán nový distribuční a konzumační model workflow napříč spravovanými repozitáři po dokončení přestavby.

3.2.3 Konfigurační model repozitáře

 **Návrh na vylepšení:** Placeholder: Zde bude specifikováno nové deklarativní schéma konfiguračního manifestu vymezujícího identitu repozitáře, doménové oblasti a projektová pravidla.

^{*}Pojem **hermetické prostředí** (angl. *hermetic environment*) označuje v softwarovém inženýrství takové výpočetní a běhové prostředí, které je zcela izolované od nekontrolovaných stavů hostitelského operačního systému a okolní sítě. Veškeré nástroje, knihovny a systémové závislosti jsou v něm explicitně deklarovány a uzamčeny na konkrétních verzích, což zaručuje absolutní determinismus a reprodukovatelnost: proces spuštěný v hermetickém kontejneru skončí vždy identickým výsledkem bez ohledu na to, kde a kdy byl vyvolán.

3.3 Životní cyklus požadavku

🔧 **Strukturální upozornění: Životní cyklus v rekonstrukci:** Stavový automat řízení požadavků, definice exekučních fází a systém validačních bran procházejí revizí v návaznosti na celkovou přestavbu DarkFactory.

Životní cyklus požadavku formalizuje přechody mezi stavy od zadání úkolu v GitHub Issues přes plánování, implementaci, automatické testování až po vystavení pull requestu a sloučení do hlavní větve.

💡 **Návrh na vylepšení:** Placeholder: Zde bude doplněn nový stavový diagram životního cyklu a detailní popis jednotlivých fází nového běhového automatu.

3.4 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček; jejich přítomnost je spolehlivější než jakýkoli ruční záznam, protože se nemůže rozejít se skutečností.

Soubor	Prostředí	Doména
pyproject.toml	Python	kód
Cargo.toml	Rust	kód
package.json	Node	kód
go.mod	Go	kód
typst.toml	Typst	text
.latexmkrc	LaTeX	text
lakefile.toml	Lean	matematika

Tabulka 3: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.4.1 Domény a prostředí

Rozlišení dvou úrovní se ukázalo jako nutné až v průběhu práce. *Prostředí* říká, jaký nástroj je potřeba; *doména* říká, jakému způsobu řízení výsledek podléhá. Python a Rust jsou dvě prostředí téže domény — kód se testuje a balí. Typst a LaTeX jsou dvě prostředí jiné domény — text se sází a publikuje.

Toto rozlišení dovolu je popsat i repozitář, který obsahuje zároveň program a text — jako právě tato práce, jejíž praktickou částí je software popisovaný v Tabulka 3. Bez něj by bylo nutné buď považovat sazbu za zvláštní případ kódu, nebo pro texty vytvořit samostatný systém.

3.4.2 Deklarace jako doplněk rozpoznávání

Rozpoznávání prostředí nemůže předvídat veškeré specifické požadavky projektu. Repozitář této práce například přibaluje vlastní sadu písem v podadresáři `fonts/`, takže příkaz

pro sazbu vyžaduje dodatečné parametry. Konfigurační manifest proto umožňuje výchozí parametry transparentně rozšířit nebo přepsat, aniž by bylo nutné vypnout automatické rozpoznávání prostředí jako celek.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedena ukázka deklarativního přepsání parametrů sestavení v novém schématu konfigurace přestavěného systému DarkFactory.

3.5 Orchestrace napříč poskytovateli

 **Strukturální upozornění: Orchestrační jádro v rekonstrukci:** Mechanismus předávání štafety, řízení kontextového okna a integrace klientských harnessů budou aktualizovány podle nového orchestračního enginu.

Systém neváže na jednoho poskytovatele modelu. Úkol je popsán způsobem nezávislým na rozhraní a předán prvnímu dostupnému poskytovateli; vyčerpá-li tento kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil. Tato vlastnost byla přímou reakcí na provozní zkušenost: zastavení celého procesu kvůli vyčerpané kvótě jediného poskytovatele bylo nejčastější příčinou prostoje.

 **Návrh na vylepšení:** Placeholder: Zde bude doplněn detailní popis nového modelu rotace, správy tokenů, bezpečnostního oddělení tajemství a bezeztrátové serializace kontextu po přestavbě systému.

3.6 Ověřování změn

 **Strukturální upozornění: Verifikační pipeline v rekonstrukci:** Způsob deterministického vyhodnocování integračních úloh a zpracování chybových stavů bude přizpůsoben nové architektuře.

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád, jak bylo vysvětleno v kapitole 2.

 **Návrh na vylepšení:** Placeholder: Zde bude popsána nová podoba verifikačních úloh, cachování závislostí a automatického reportování incidentů po přestavbě systému.

3.7 Projektová automatizace a správa nastavení jako kód

 **Strukturální upozornění: Projektová automatizace v rekonstrukci:** Nástroje pro synchronizaci GitHub Projects, nastavení repozitáře a ochranu větví procházejí rekonstrukcí.

Automatizace repozitáře zajišťuje auditovatelnost a konzistentní správu projektových pravidel bez nutnosti manuálních zásahů v rozhraní platformy.

 **Návrh na vylepšení:** Placeholder: Zde bude popsána nová implementace projektové synchronizace a správy repozitářových pravidel jako kód v přestavěném systému DarkFactory.

3.8 Automaticky generovaná dokumentace

 **Strukturální upozornění: Dokumentační subsystém v rekonstrukci:** Generování a validace živé dokumentace budou přizpůsobeny novému rozhraní systému DarkFactory.

V moderním softwarovém vývoji představuje manuální údržba dokumentace permanentní zdroj chyb a desynchronizace: jakmile se kód vyvíjí rychleji než textové popisy, dokumentace se nevyhnutelně stává zastaralou a nespolehlivou. Systém DarkFactory proto prosazuje striktní princip stoprocentního odvozování veškeré projektové a API dokumentace přímo ze zdrojového kódu a strukturovaných inline komentářů (tzv. *living documentation*).

 **Návrh na vylepšení:** Placeholder: Zde bude doplněno nové procesní schéma a popis generování a striktní validace živé dokumentace v CI pipeline.

3.9 Systém revizních značek pro lidský dohled nad akademickým textem

Při rozšiřování systému DarkFactory na tvorbu a revizi akademických a odborných textů (doména textu) vyvstala potřeba formalizovat spolupráci člověka a autonomního agenta přímo v sazebním formátu Typst. Výsledkem je protokol vizuálních revizních značek (**Review Markers**) a textových revizních funkcí, který barevně a sémanticky rozlišuje stav zpracování jednotlivých pasáží:

 **Návrh na vylepšení:** Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

 **Chyba / Nesrovnalost k opravě:** Detekované věcné nepřesnosti, logické mezery, překlepy nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

 **Strukturální upozornění:** Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

 **Hloubková kritika / Oponentura:** Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabin formulovaných jako břitké otázky k obhajobě.

V toku textu se uplatňují tyto zvýrazňovací a srovnávací funkce:

- **Žluté zvýraznění** (`#draft[...]` / `#unconfirmed[...]`): Označuje neověřený text konceptu čekající na autorské posouzení a revizi.

- Zelené zvýraznění (`#added[...]`): Označuje nově přidaný text vygenerovaný autonomním agentem na základě požadavku či doporučení.
- Modré zvýraznění (`#confirmed[...]`): Označuje text potvrzený uživatelem, který dosud neprošel finální integrací.
- Červené zvýraznění s přeškrtnutím (`#removed[...]`): Označuje text navržený k odstranění z rukopisu.
- ~~Původní nahrazovaný text~~ Srovnávací diff (`#diff(old, new)`): Zobrazuje původní text přeškrtnutý v červené barvě následovaný novým textem v zelené barvě.
- Čistý neoznačený text představuje finální, autorsky schválený a přijatý text v hlase autora (v rozpracovaném stavu konceptu jsou veškeré dosud neuzavřené pasáže zviditelněny revizními funkcemi).

Tento protokol umožňuje autonomnímu agentovi navrhopvat změny s transparentním vyznačením míry jistoty a člověku poskytuje okamžitou vizuální kontrolu nad tím, které části rukopisu již prošly lidskou redakcí a které ještě čekají na posouzení.

4 Výsledky a diskuse

 **Strukturální upozornění: Výsledky a empirická data v rekonstrukci:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory byla detailní provozní data předchozí verze (původní statistika 61 zpracovaných úkolů, měření spotřeby tokenů a rozbor konkrétních chyb ve starých skriptech) dočasně nahrazena zástupnými strukturálními bloky. Po dokončení a stabilizaci nové verze proběhne nová série měření a testování.

4.1 Profil nasazených repozitářů a testovací prostředí

Pro praktické ověření funkčnosti, robustnosti a přenositelnosti systému DarkFactory bylo zvoleno testovací prostředí skládající se ze tří produkčních repozitářů s odlišnou architekturou, programovacími jazyky a rozsahem:

1. **DarkFactory** (mateřský repozitář): Centrální vývojové prostředí řídicí pipeline a sdílených orchestračních nástrojů.
2. **OdbornaPrace-paper**: Repozitář samotného rukopisu této práce reprezentující doménu akademických a odborných textů. Kódová báze kombinuje sazbu v moderním typografickém systému Typst, doprovodné obslužné skripty a plně automatizovanou kompilaci a publikaci na GitHub Pages.
3. **ChessWithQuests**: Aplikační projekt implementující šachovou herní logiku a pravidla. Slouží k ověření chování pipeline při izolovaných, algoritmicky ohraničených úlohách s rychlou zpětnou vazbou jednotkových testů.

 **Návrh na vylepšení:** Placeholder: Zde bude po přestavbě specifikováno aktualizované běhové prostředí nové verze systému DarkFactory.

4.2 Sjednocení pracovních postupů

Základním architektonickým principem systému DarkFactory je sdílení pracovních postupů namísto jejich ad-hoc kopírování do jednotlivých projektů. Klientské repozitáře neudržují vlastní izolované sady validačních skriptů, linterů ani integračních definic; namísto toho delegují exekuci na centrální znovupoužitelné šablony a veškerou projektovou specifičnost deklarují v jediném manifestu.

Skutečný inženýrský přínos sjednocení nespočívá v pouhé kvantitativní redukci řádků kódu (LOC), nýbrž v kvalitativních a architektonických vlastnostech:

- **Údržbová složitost** $O(1)$: Před sjednocením vyžadovala jakákoli změna v CI procesu samostatnou manuální úpravu a pull request v každém repozitáři ($O(N)$). Po sjednocení je oprava provedena pouze jednou v upstreamovém repozitáři DarkFactory a spotřebitelské projekty ji přebírají posunem připnuté verze v manifestu.

- **Statická kontrola a testovatelnost:** Centralizované jádro metaharnessu podléhá striktní typové kontrole a je pokryto sadou testů spouštěných lokálně.
- **Omezení jediného bodu selhání (SPOF):** Riziko nechtěných změn je deterministicky eliminováno verzováním: klientské repozitáře odkazují na neměnný kryptografický SHA hash commitu či sémantický tag.
- **Konzistentní agentní prostředí:** Zajištění, že životní cyklus požadavku, správa kontextu, rotace poskytovatelů i vyhodnocování validačních bran probíhají ve všech repozitářích exaktně stejným způsobem.

4.3 Doménové ověření na sazbě textu

Praktickým ověřením obecnosti systému byla sazba samotného rukopisu této práce: repozitář je detektorem automaticky klasifikován do domény textu, v rámci integračního běhu v GitHub Actions se provede kompilace přes nativní binárku Typst, vygenerované PDF je uloženo jako auditovatelný artefakt běhu a současně automaticky publikováno na dokumentační portál GitHub Pages. Empirické měření v CI runneru prokázalo vysokou efektivitu moderního sazebního systému: kompletní kompilace celého dokumentu včetně načtení písem, vektorových schémat a bibliografie trvá v průměru pouhé 1–2 sekundy.

4.4 Provozní metriky a spolehlivost agentních běhů

 **Strukturální upozornění: Metriky čekají na novou evaluaci:** Původní empirická data ze staré verze systému byla vyřazena. Po dokončení přestavby systému DarkFactory proběhne nová série benchmarků a provozních měření nad reálnými vývojovými požadavky.

 **Návrh na vylepšení:** Placeholder: Zde bude po dokončení přestavby zařazena nová tabulka empirických metrik (počet zpracovaných issues, úspěšnost PR na první pokus, úspěšnost po automatické samoopravě, průměrná spotřeba tokenů, časová latence a četnost rotace modelů).

4.5 Poznatky a analýza chyb z provozu

 **Strukturální upozornění: Provozní poznatky v revizi:** Konkrétní chyby dřívějšího skriptového jádra budou nahrazeny poznatky a incidenty z provozu přestavěného systému.

Dosavadní provoz odhalil, že nejkritičtější úskalí autonomních vývojových provozů neleží v neschopnosti modelů generovat syntakticky správný kód, nýbrž v distribuovaném řízení a systémové orchestraci:

- **Řízení souběžnosti a zámky:** Koordinace volajících a volaných úloh vyžaduje striktní hierarchizaci, aby nedocházelo k uvážnutí workflow.
- **Mapování názvů kontrol a branch protection:** Pravidla ochrany větví musí přesně odpovídat složeným názvům úloh emitovaným CI platformou.

- **Transparentnost hlášení selhání:** Systém nesmí tlumit výjimky a maskovat chyby; každé selhání API musí být transparentně zachyceno a eskalováno.
- **Dynamická detekce prostředí:** Systém nesmí spoléhat na implicitní předpoklady o technologiích v repozitáři, ale odvozovat exekuční plán z deklarativních manifestů.

 **Návrh na vylepšení:** Placeholder: Zde bude po přestavbě doplněn podrobný rozbor konkrétních chyb a jejich deterministických technických řešení v nové architektuře DarkFactory.

4.6 Diskuse

Nejpoučnejším z provozních zkušeností je zjištění, že u autonomního vývojového systému je mechanismus detekce a transparentního hlášení selhání stejně kritický jako samotná schopnost generovat kód. Tiché ignorování chyb vytváří iluzi fungujícího provozu a eroduje důvěru člověka v autonomní proces. Druhým klíčovým poznatkem je nutnost eliminace implicitních předpokladů o repozitáři: prostředí musí být detekováno z deklarativních manifestů za běhu, nikoli pevně zadrátováno v šablonách. Zásadní otázkou je, jak si koncepce systému DarkFactory stojí ve srovnání se současnými agentními vývojovými platformami (např. SWE-agent (6), Devin či GitHub Copilot Workspace):

- **Deterministický DAG vs. nekonečná agentní smyčka:** Systémy jako SWE-agent spouštějí jediný monolitický model v interaktivní terminálové smyčce, kde model sám rozhoduje o ukončení práce. Pokud model uvízne v bludném kruhu nebo halucinuje, snadno vyčerpá celý rozpočet tokenů. DarkFactory naproti tomu uzavírá model do deterministického grafu v GitHub Actions: každý krok (plánování, implementace, verifikace) má striktně alokovaný rozpočet tahů a samostatnou časovou i kontextovou izolaci.
- **Granularita lidského dohledu:** Většina komerčních nástrojů staví na paradigmatu „jedno zadání → finální pull request“. Pokud agent na začátku špatně pochopí záměr, stráví minuty generováním stovek řádků nepoužitelného kódu, jehož následná revize vývojáře vyčerpává (*review fatigue*). DarkFactory toto riziko eliminuje vícefázovým schvalováním: člověk schvaluje záměr, konkrétní technický plán a teprve následně je spuštěn kódovací agent.
- **Bezpečnost a hermetičnost:** Zatímco běh autonomního agenta přímo na lokálním vývojovém stroji přináší riziko nechtěného poškození konfigurace nebo úniku citlivých environmentálních proměnných, DarkFactory běží v efemérních kontejnerech CI platformy s oddělenými právy tokenů a auditovatelným logem každého provedeného příkazu.

4.7 Omezení

Výsledky a dosažená zjištění je třeba interpretovat s ohledem na tři fundamentální omezení navrženého řešení:

1. **Hranice deterministické verifikovatelnosti:** Systém vyžaduje, aby bylo správnost navržené změny možné objektivně ověřit automatizovanými testy, lintery či kompilátorem. U úloh subjektivní či kreativní povahy — jako je ergonomie uživatelského rozhraní, ladění vizuálních stylů nebo stylistická formulace odborného textu — zůstává autonomní přínos omezen na vygenerování prvotního návrhu, jehož validaci musí provést člověk.
2. **Propustnost a dostupnost inferenčních API:** Ačkoli rotační žebříček poskytovatelů minimalizuje dopad výpadku jedné služby, celková průchodnost pipeline je limitována globálními kvótami a latencí cloudových API. Při souběžném zpracování většího množství požadavků může dojít k vyčerpání všech dostupných účtů, což pipeline dočasně pozastaví.
3. **Rozsah a profil testovacího prostředí:** Ověření probíhalo na repozitářích spravovaných autorem práce. Výsledky tudíž nezachycují specifika rozsáhlých distribuovaných týmů, kde do hry vstupují merge konflikty při souběžné práci desítek vývojářů nad stejnými moduly, komplexní organizační schvalovací procesy a přísné bezpečnostní audity třetích stran.

5 Závěr

 **Strukturální upozornění: Závěr a vyhodnocení cílů v revizi:** Vzhledem k probíhající zásadní přestavbě systému DarkFactory jsou konkrétní empirické statistiky předchozí verze vyřazeny a nahrazeny strukturálními zástupnými bloky. Finální vyhodnocení a zodpovězení výzkumných otázek bude aktualizováno po dokončení evaluace nové verze.

Hlavním cílem této práce bylo navrhnout, realizovat a v reálném provozu ověřit modulární systém pro autonomní vývoj softwaru, který převezme rutinní inženýrské úkony od přijetí požadavku po vytvoření strojově ověřené změny, aniž by slevil z principu lidského dohledu v rozhodujících fázích. V rámci inženýrského cyklu byl navržen systém DarkFactory a nasazen na testovací repozitáře zahrnující jak programovací kód, tak akademickou sazbu. Naplnění jednotlivých dílčích cílů lze ve vztahu ke stanovené metodice shrnout následovně:

1. **Současný stav a teoretická východiska (Kapitola 2):** Práce systematicky zmapovala architekturu moderních dekodérových transformerů, mechanismus pozornosti, limity kontextového okna (včetně jevu *Context Rot* a významu KV cache), techniky inženýrství promptů a formální strukturu autonomní ReAct smyčky alternující vnitřní rozvahu (*Thought*) a volání nástrojů (*Action*).
2. **Architektonický návrh (Kapitola 3):** Byl navržen modulární model propojující platformu GitHub, řídicí orchestrační vrstvu a izolované běhové prostředí pro modely. Životní cyklus požadavku byl formalizován do stavového diagramu s explicitními schvalovacími branami.
3. **Realizace a nasazení (Kapitola 3 a 4):** Systém byl nasazen a ověřen na repozitářích různého zaměření — vlastní vývojové prostředí systému, textový repozitář této odborné práce (`OdbornaPrace-paper`) a aplikační projekt `ChessWithQuests` .
4. **Vyhodnocení a provozní analýza (Kapitola 4):** Architektonické sjednocení na sdílené pracovní postupy a centrální manifest eliminovalo roztříštěnou údržbu jednoúčelových skriptů napříč projekty s údržbovou složitostí $O(1)$. Provozní zkušenosti potvrdily, že klíčová úskalí neleží v generování kódu modely, nýbrž v distribuované orchestraci, řízení souběžnosti a dynamické detekci prostředí.

V návaznosti na výsledky lze zodpovědět stanovené výzkumné otázky:

- **Zodpovězení VO1 (Míra automatizace a role člověka):** Vývojový proces od zadání v GitHub Issues po vytvoření ověřeného pull requestu lze úspěšně zautomatizovat. Člověk v průběhu řešení nepsal ani neupravoval zdrojový kód, nýbrž působil výhradně jako architektonický dozor v rámci vícefázového schvalování a finální revize pull requestu.

 **Návrh na vylepšení:** Placeholder: Zde bude doplněna přesná kvantitativní úspěšnost po evaluaci nové verze DarkFactory.

- **Zodpovězení VO2 (Doménová přenositelnost):** Systém prokázal přenositelnost centrálního workflow mezi odlišnými doménami. Repozitář této odborné práce (`OdbornaPrace-paper`) využívá tutéž agentní pipeline jako softwarové projekty; veškerá doménová specifika (nástroj Typst, parametry sazby) jsou zapouzdřena v deklarativní konfiguraci bez nutnosti větvit sdílené šablony.
- **Zodpovězení VO3 (Provozní odolnost a obnova):** Víceúrovňová rotace kvót a kontrolní body zabránily haváriím integračních běhů při vyčerpání limitů API. Integrovaná smyčka samooprav navíc dokázala autonomně vyřešit syntaktické a integrační regrese na základě zpětné vazby z testů.

 **Návrh na vylepšení:** Placeholder: Zde budou doplněny konkrétní statistiky obnovy a spolehlivosti nové verze systému.

Klíčové architektonické principy a přínosy systému DarkFactory:

- **Vícefázový Human Gate:** Oddělení schvalování záměru a technického plánu v GitHub Issues před samotným zásahem do kódu zaručuje lidskou kontrolu nad architektonickými mantinely a šetří čas vývojáře.
- **Orchestrace nezávislá na poskytovateli:** Pipeline eliminuje závislost na jediném poskytovateli jazykových modelů mechanismem rotace účtů a modelových fondů při vyčerpání limitů.
- **Bezeztrátové předávání štafety:** Uložení rozpracovaného stavu a kontextu do kontrolního bodu umožňuje plynulé navázání náhradního modelu bez ztráty dosavadní práce.
- **Jediný zdroj pravdy:** Veškeré odchylky repozitáře jsou deklarovány v konfiguračním manifestu, což udržuje sdílenou infrastrukturu snadno spravovatelnou.

 **Návrh na vylepšení:** Placeholder: Zde budou specifikovány nové technické inovace přestavěného systému DarkFactory.

Další rozvoj systému DarkFactory se nabízí ve čtyřech perspektivních směrech:

1. **Formální verifikace:** Rozšíření aplikačních domén o interaktivní dokazovače vět a formální matematické prostředí (jazyk Lean 4).
2. **Autonomní samoléčení:** Automatické generování izolovaných regresních testů přímo ze stack trace chyb v CI a pověření agenta jejich okamžitou nápravou.
3. **Hybridní inferenční vrstva:** Zapojení lokálních open-weights modelů jako bezplatného prvního stupně pipeline pro rutinní syntaktickou kontrolu před delegováním komplexních úloh na velká cloudová API.
4. **Kryptografická bezpečnost dodavatelského řetězce:** Zavedení automatického podepisování commitů vygenerovaných agentem pomocí klíčů GPG či Sigstore pro nezpochybnitelnou provenienci kódu.

Seznam příloh

A	Obsah přiloženého média	41
B	Schéma konfiguračního manifestu darkfactory.json	42
C	Sdílené workflow pro GitHub Actions	43
D	Systémové prompty plánovacího a kódovacího agenta	44
E	Protokol revizních značek v sazebním systému Typst	45
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	45

KONCEPT

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ             # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/references.bib                # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/                         # Pomocné a náhledové skripty
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/               # Sdílené šablony kontinuální
integrace
│   └─ src/                             # Výkonné orchestrační jádro systému
├─ out/prace.pdf                        # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 1: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

 **Strukturální upozornění: Příloha v rekonstrukci:** Formální JSON schéma manifestu bude doplněno po stabilizaci nového schématu v přestavěném systému DarkFactory.

 **Návrh na vylepšení:** Placeholder: Zde bude uvedeno úplné JSON Schema vymezující validní syntaxi nového konfiguračního manifestu.

KONCEPT

C Sdílené workflow pro GitHub Actions

 **Strukturální upozornění: Příloha v rekonstrukci:** Definice sdílených a volajících workflow pro GitHub Actions budou aktualizována po dokončení přestavby orchestrátoru.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny ukázky znovupoužitelného a volajícího workflow nové verze DarkFactory.

KONCEPT

D Systémové prompty plánovacího a kódovacího agenta

 **Strukturální upozornění: Příloha v rekonstrukci:** Systémové prompty pro plánovací a implementační fáze budou aktualizovány podle nových rolí a instrukčních šablon v přestavěném systému.

 **Návrh na vylepšení:** Placeholder: Zde budou uvedeny kompletní systémové prompty pro jednotlivé fáze životního cyklu požadavku.

KONCEPT

E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v sazebním formátu Typst.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 2: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

E.1 Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břítka, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabín metodiky a příprava na otázky zkušební komise.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** (`#draft` / `#unconfirmed`): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** (`#added`): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** (`#confirmed`): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** (`#removed`): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** (`#diff`): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Každá značka plní přesně vymezenou komunikační roli v procesu lidského schvalování: zatímco finální text zůstává zcela bez zvýraznění, veškeré neověřené pasáže konceptu jsou zřetelně žluté (`#draft`), nově přidané části zelené (`#added`), potvrzené části modré (`#confirmed`) a opravy zviditelněné přes srovnávací diff (`#diff`). Náměty, chyby, strukturní vady i britká kritika jsou navíc striktně separovány do barevných postranních panelů na okraji textu.

Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
5. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
6. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
7. XIAO, Guangxuan, TIAN, Yuandong, CHEN, Beidi, HAN, Song a LEWIS, Mike. Efficient Streaming Language Models with Attention Sinks. *arXiv preprint arXiv:2309.17453*. Online. 2023. Available from: <https://arxiv.org/abs/2309.17453>
8. ZHANG, Zhenyu, SHENG, Ying, ZHOU, Tianyi, CHEN, Tianlong, ZHENG, Lianmin, CAI, Ruisi, SONG, Zhao, TIAN, Yuandong, RÉ, Christopher, BARRETT, Clark, WANG, Zhangyang a CHEN, Beidi. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 34661–34730. Available from: <https://arxiv.org/abs/2306.14048>
9. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
10. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

11. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
12. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
13. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
14. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>